



DOCUMENTAÇÃO DO DESENVOLVEDOR

Sistema de Gerenciamento de Usuários, Pets e Empresas

Disponível em: <<https://github.com/mlrlima/tela-login-angular/tree/v2>>

Versão 2.0
Agosto de 2026

Sumário

Sumário.....	2
1. Visão Geral.....	3
2. Arquitetura.....	3
3. Stack Tecnológica.....	4
4. Estrutura de Pastas.....	5
5. Modelo de Dados.....	6
6. Camadas do Back-end.....	7
6.1 Controllers (@RestController).....	7
6.2 Services (@Service).....	7
6.3 Repositories (Spring Data JPA).....	7
6.4 DTOs (pacote dto).....	8
6.5 Config.....	8
6.6 Exception Handling.....	8
6.7 WebSocket (pacote websocket) — mapa de pets em tempo real.....	9
7 Autenticação e Autorização.....	10
7.1 Regras de acesso (SecurityConfig).....	10
7.2 TokenService — geração e validação de JWT.....	11
8. Referência da API REST.....	11
8.1 Autenticação — /auth.....	11
8.2 Usuários — /usuario.....	11
8.3 Pets — /pet.....	12
8.4 Empresas — /empresa.....	12
8.5 WebSocket (tempo real).....	13
9. Front-end Angular.....	13
9.1 Rotas (app.routes.ts).....	13
9.2 Autenticação no front.....	13
9.3 Services.....	14
9.4 Configuração de ambiente.....	14
10. Como Rodar.....	14
11. Banco de Dados.....	14
12. Limitações Conhecidas e Próximos Passos.....	15
12.1 Pontos novos identificados nesta revisão.....	15

Nota desta revisão (2.0): Este documento foi conferido diretamente contra o código-fonte atual da branch v2 (não apenas contra a versão anterior desta documentação). Diversas informações da versão anterior estavam desatualizadas — por exemplo, o sistema de autenticação por UUID em memória foi substituído por Spring Security + JWT em cookie httpOnly, e três das cinco "limitações conhecidas" já foram corrigidas no código atual. As seções abaixo refletem o estado real do repositório; os pontos que mudaram em relação à versão anterior estão sinalizados.

1. Visão Geral

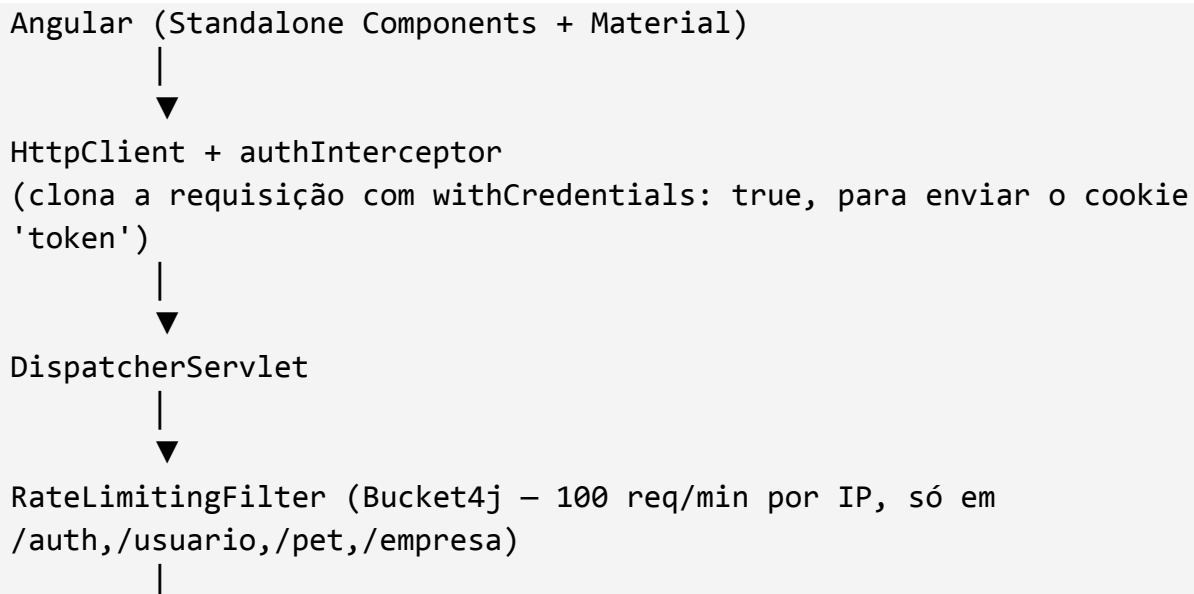
O PetManager é uma aplicação web full-stack para gestão de usuários, pets e empresas:

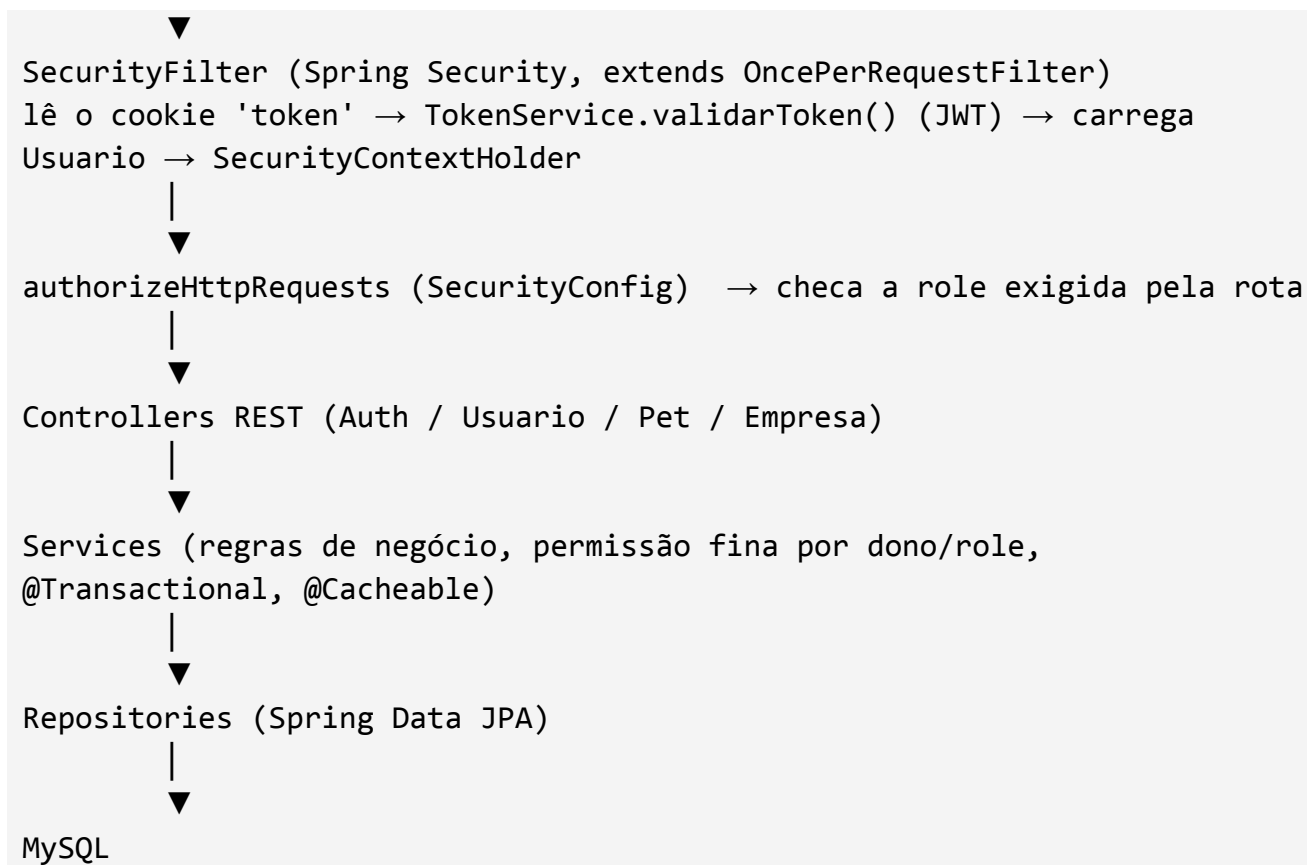
- Back-end: API REST em Spring Boot, empacotada como WAR e implantada em um Tomcat 11 externo.
- Front-end: Angular (standalone components) com Angular Material, consumindo a API via HttpClient.
- Persistência: MySQL via Spring Data JPA.
- Segurança: Spring Security com autenticação stateless por JWT, transportado em cookie httpOnly, e controle de acesso por role (ADMIN, USER).
- Tempo real: WebSocket (STOMP sobre SockJS) para transmitir a localização dos pets em um mapa ao vivo.

Em produção, o Angular é compilado e servido pelo próprio Tomcat, no mesmo contexto da API (/tela-login-angular); um SpaFallbackController redireciona as rotas do Angular para o index.html.

2. Arquitetura

Fluxo de requisição:





Padrão em camadas: controller → service → repository → model, com pacotes de apoio security (agora baseado em Spring Security), config, dto, exception e websocket.

Mudança em relação à versão anterior: A versão anterior descrevia um AuthInterceptor customizado que validava um token UUID guardado em Map (ConcurrentHashMap) e uma anotação própria @Secured. Isso não existe mais no código atual: a branch v2 usa Spring Security de verdade (SecurityFilterChain, UserDetails, SecurityContextHolder) com JWT assinado (biblioteca java-jwt, da Auth0) transportado em cookie httpOnly — não mais em header Authorization nem em localStorage.

3. Stack Tecnológica

Camada	Tecnologia
Linguagem (back)	Java 17
Framework (back)	Spring Boot (web, data-jpa, validation, tomcat)
Segurança	Spring Security + JWT (java-jwt / Auth0, cookie httpOnly)
Rate limiting	Bucket4j (bucket4j_jdk17-core)

Cache	Spring Cache (spring-boot-starter-cache, SimpleCacheManager em memória)
Tempo real	Spring WebSocket + STOMP (SockJS no front)
Namespace	Jakarta (jakarta.persistence, jakarta.validation, jakarta.servlet)
ORM / dados	Spring Data JPA + Hibernate
Banco	MySQL (mysql-connector-j 9.3.0)
Build (back)	Maven · empacotamento WAR (finalName: tela-login-angular)
Servidor	Tomcat 11 externo
Front-end	Angular (standalone components), Angular Material, TypeScript
Build (front)	Angular CLI (ng serve / ng build)

Sobre versões antigas: Versões anteriores deste projeto usaram Spring MVC puro + Weld/CDI, e depois um sistema próprio de token UUID. Nenhum dos dois existe mais — o back-end atual é 100% Spring Boot + Spring Security. Ao encontrar menções a "Spring MVC", "CDI", "AuthInterceptor" ou "@Secured" em textos antigos, considere desatualizado.

4. Estrutura de Pastas

```
tela-login-angular/ (branch v2)
├── pom.xml                # build do back-end (Spring Boot,
WAR)
├── package.json           # script "dev" (roda back + front
juntos)
├── src/main/java/
│   ├── config/            # bootstrap, cache
│   ├── controller/        # REST + SPA fallback
│   ├── dto/               # DTOs de resposta
│   ├── exception/         # handler e ErrorResponse
│   ├── model/             # entidades
│   ├── repository/        # Spring Data JPA
│   ├── security/          # SecurityConfig, filtros, JWT, Auth
│   ├── service/           # regras de negócio
│   └── websocket/         # STOMP, buffer e broadcaster
├── src/main/resources/    # application.properties, data.sql
└── casos-de-teste/        # Postman + simulador WebSocket
```

```
└─ angular-frontend/src/app/      # componentes, services, guards,
interceptor e rotas
```

Novidades de pastas na v2: Em relação à versão anterior: pacote `dto/`, `exception/`, `security/` reestruturado e novo pacote `websocket/`. No front, entraram `gestao-empresas/`, `empresa-form/`, `mapa-pets/`, `pet-mapa-dialog/` e os novos services relacionados.

5. Modelo de Dados

Usuario N:N Empresa (empresa_usuario)

1:N



Pet

Usuario: id, email unique, senha BCrypt, nome, role, empresas

Empresa: id, nome unique, usuarios

Pet: id, nome, dono, especie, latitude, longitude, intervaloMover, dataNascimento, peso

Role: ADMIN, USER

Especie: CACHORRO, GATO, PEIXE, ROEDOR, AVE, OUTRA

Usuario — tabela usuario; email é único; implementa UserDetails (Spring Security), expondo role como GrantedAuthority (ROLE_ADMIN também recebe ROLE_USER); relação N:N com Empresa.

Empresa — tabela empresa; nome é único; relação N:N com Usuario via tabela associativa empresa_usuario.

Pet — tabela pet; relação N:1 com Usuario via FK user_id; especie persistida como String (@Enumerated(EnumType.STRING)); campos latitude/longitude/intervaloMover suportam o mapa em tempo real; dataNascimento e peso são informações cadastrais do pet.

Todos os id são gerados com GenerationType.IDENTITY (auto-incremento); equals/hashCode baseados no id.

O que mudou no modelo desde a versão anterior: Empresa é nova, com relação N:N com Usuario. Pet recebeu latitude, longitude, intervaloMover, dataNascimento e peso. PASSARO foi renomeado para AVE. A senha agora é armazenada com hash BCrypt.

6. Camadas do Back-end

6.1 Controllers (@RestController)

Expõem os endpoints e delegam para os services; não contêm regra de negócio. Exemplo — PetController:

```
@GetMapping("/all")
public ResponseEntity<Page<PetResponseDTO>> getAll(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size) {
    Pageable pageable = PageRequest.of(page, size);
    return ResponseEntity.ok(service.getAllPets(pageable));
}
```

6.2 Services (@Service)

Concentram regra de negócio, verificação de permissão (dono/role), transações (@Transactional) e cache (@Cacheable/@CacheEvict). Exemplo — PetService:

```
@Cacheable(value = "pets", key = "#pageable.pageNumber + '-' +
#pageable.pageSize",
    condition = "#root.target.isAdminLogado()")
public Page<PetResponseDTO> getAllPets(Pageable pageable) {
    Usuario usuarioLogado = logado();
    Page<Pet> pets = usuarioLogado.getRole() == Role.ADMIN
        ? petRepository.findAll(pageable)
        : petRepository.findByDono_Id(pageable, usuarioLogado.getId());
    return pets.map(this::toDTO);
}
```

Todos os services expõem um método privado `logado()`, que lê o usuário autenticado via `SecurityContextHolder.getContext().getAuthentication().getPrincipal()` (o próprio `Usuario`, pois implementa `UserDetails`).

6.3 Repositories (Spring Data JPA)

Interfaces que estendem `JpaRepository<Entidade, Long>`; o Spring gera a implementação e as queries a partir do nome do método:

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    Usuario findByEmail(String email);
}
```

```

public interface PetRepository extends JpaRepository<Pet, Long> {
    Page<Pet> findByDono_Id(Pageable pageable, Long donoId);
    List<Pet> findAllByDono_Id(Long donoId);
}

public interface EmpresaRepository extends JpaRepository<Empresa, Long> {
    List<Empresa> findByUsuariosId(Long usuarioId);
    Optional<Empresa> findByNome(String nome);
}

```

6.4 DTOs (pacote dto)

Cada entidade tem um *ResponseDTO próprio para as respostas da API, evitando expor o campo senha e evitando loop infinito de serialização (Usuario ↔ Empresa ↔ Usuario...):

- UsuarioResponseDTO: id, email, nome, role, empresas (lista de EmpresaRelacionadaDTO — apenas id/nome).
- PetResponseDTO: id, nome, especie, dono (DonoDTO — apenas id/email), latitude, longitude, intervaloMover, dataNascimento, peso.
- EmpresaResponseDTO: id, nome, usuarios (lista de UsuarioRelacionadoDTO — apenas id/email).
- LocalizacaoPetDTO: latitude/longitude com @NotNull, usado no corpo do PUT /pet/{id}/localizacao.

Débito resolvido: A versão anterior apontava a exposição da entidade Usuario completa, inclusive senha, em endpoints de leitura. Isso está corrigido: os endpoints retornam DTOs sem o campo senha.

6.5 Config

- Application: bootstrap Spring Boot e suporte a WAR.
- CachingConfig: SimpleCacheManager com caches usuarios, usuarioPorId, empresas e pets em memória.

6.6 Exception Handling

GlobalExceptionHandler (@ControllerAdvice) centraliza o tratamento de erros.

Exceção	HTTP	Uso típico
ResourceNotFoundException	404	Usuário, pet ou empresa não encontrado
UnauthorizedException	401	Sem permissão e senha inválida
ConstraintViolationException	400	Falha de validação Bean Validation

Exception (genérica)	500	Erro não mapeado; detalhes ficam no log do servidor
----------------------	-----	---

Observação: Casos de usuário autenticado, mas sem direito sobre o recurso, semanticamente seriam HTTP 403 (Forbidden), enquanto 401 é mais apropriado para não autenticado. O código atual ainda usa 401 para esses casos.

6.7 WebSocket (pacote websocket) — mapa de pets em tempo real

Pacote inteiramente novo em relação à versão anterior do documento. Implementa o envio em tempo real da localização dos pets para quem estiver com o mapa aberto no front-end, usando STOMP sobre WebSocket (com fallback SockJS).

- **WebSocketConfig:** Habilita o broker de mensagens do Spring (@EnableWebSocketMessageBroker) e registra o endpoint de conexão.
- **PetLocationBuffer:** Buffer thread-safe (Map<Long, PetResponseDTO> sobre ConcurrentHashMap) que acumula as posições atualizadas de cada pet até o próximo envio em lote. Como a chave é o id do pet, se o mesmo pet mandar várias atualizações dentro do mesmo intervalo, só a última posição é mantida.
- **PetLocationBroadcaster:** Um job agendado (@Scheduled) que roda a cada 500ms, drena o buffer e, se houver algo pendente, envia o lote para o canal /topic/pet-location-lote, para quem estiver inscrito.

Como as três classes se conectam ao restante do sistema:

1. O front-end chama PUT /pet/{id}/localizacao (endpoint REST comum, protegido por autenticação normal).
2. PetController.atualizarLocalizacao() delega a validação/gravação para PetService.atualizarLocalizacao() (verifica dono/ADMIN, salva no banco) e, em seguida, chama PetLocationBuffer.adicionar() com o DTO já atualizado.
3. Em paralelo, o PetLocationBroadcaster (rodando independentemente, a cada 500ms) drena o que estiver acumulado no buffer e publica em /topic/pet-location-lote.
4. Todo cliente Angular inscrito nesse tópico (websocket-service.ts) recebe o lote e atualiza o mapa (petLocationBatch\$).

Por que agrupar em lote em vez de enviar direto? Se cada atualização de posição disparasse uma mensagem WebSocket imediatamente, N pets se movendo ao mesmo tempo gerariam N mensagens por instante. O buffer + broadcaster agendado agrupa tudo que mudou em uma janela de 500ms em

uma única mensagem, reduzindo o tráfego e a carga de processamento no cliente — um padrão de agregação ("debounce em lote") comum em sistemas de posição em tempo real.

Limitação: também é só em memória Assim como o token/blacklist e os caches, o PetLocationBuffer vive em ConcurrentHashMap — não é compartilhado entre instâncias do backend. Em uma implantação com múltiplas instâncias atrás de um load balancer, um cliente conectado à instância A não receberia atualizações originadas por uma escrita que caiu na instância B, a menos que se introduza um broker externo (ex.: RabbitMQ/Redis Pub-Sub) no lugar do enableSimpleBroker em memória.

7 Autenticação e Autorização

Fluxo de um request de login e de um request protegido:

1. POST /auth/login {email, senha}
AuthenticationManager → AuthService → UsuarioRepository
→ BCrypt → TokenService.gerarToken(usuario)
→ JWT HMAC256, issuer 'tela-login-angular', expiração de 30 min
→ Set-Cookie: token=<jwt>; HttpOnly; Secure; SameSite=Strict;
Max-Age=1800
→ LoginResponseDTO { id, nome, email, role }
2. Request protegido
→ navegador envia cookie token
→ SecurityFilter valida JWT e carrega Usuario
→ SecurityContextHolder recebe Authentication
→ SecurityConfig verifica role
→ Controller/Service executa

7.1 Regras de acesso (SecurityConfig)

Rota	Regra
POST /auth/login	Público
POST /auth/usuario	Público
/empresa/**	Somente ROLE_ADMIN
Assets Angular	Públicos
Fallback SPA	Público

/ws/**	Público
Demais rotas	Exigem autenticação

A checagem fina — "é o dono deste pet?", "é ADMIN ou é o próprio usuário?" — não está no SecurityConfig; é feita manualmente dentro dos Services, olhando o usuário autenticado (SecurityContextHolder) contra o dono do recurso.

7.2 TokenService — geração e validação de JWT

- gerarToken(usuario): JWT HMAC256, issuer fixo, subject = e-mail, expiração em 30 minutos.
- validarToken(token): verifica assinatura e issuer e retorna o subject.
- invalidarToken/tokenEstaValido: blacklist em memória para logout.

Cookie Secure e desenvolvimento local: O cookie usa .secure(true), portanto exige HTTPS para ser reenviado pelo navegador. Para desenvolvimento HTTP local, pode ser necessário usar temporariamente .secure(false), nunca em produção.

Blacklist de logout também é em memória: A blacklist é perdida no restart e não é compartilhada entre múltiplas instâncias. A expiração curta de 30 minutos reduz o impacto, mas a limitação deve ser conhecida.

8. Referência da API REST

As rotas reais partem de /auth, /usuario, /pet e /empresa, sob o context-path do WAR (/tela-login-angular em produção).

8.1 Autenticação — /auth

Método	Rota	Protegido	Descrição
POST	/auth/login	Não	Login; seta cookie JWT e retorna LoginResponseDTO
POST	/auth/logout	—	Expira cookie e adiciona token à blacklist
POST	/auth/usuario	Não	Cadastro público; role USER; BCrypt

8.2 Usuários — /usuario

Método	Rota	Regra extra
--------	------	-------------

GET	/usuario/all?page&size	ADMIN: todos; USER: somente ele
GET	/usuario/{id}	ADMIN ou próprio usuário
GET	/usuario?email=	ADMIN ou próprio usuário
PUT	/usuario	ADMIN ou próprio; senha em branco mantém atual
DELETE	/usuario/{id}	ADMIN ou próprio; cascata em pets e desvincula empresas

8.3 Pets — /pet

Método	Rota	Regra extra
GET	/pet/all?page&size	ADMIN: todos; USER: somente seus pets
GET	/pet/{id}	Dono ou ADMIN
POST	/pet	Dono é o usuário autenticado
PUT	/pet	Dono ou ADMIN; não troca dono
DELETE	/pet/{id}	Dono ou ADMIN
PUT	/pet/{id}/localizacao	Dono ou ADMIN; atualiza buffer WebSocket
GET	/pet/{id}/intervalo	Dono ou ADMIN

8.4 Empresas — /empresa

Método	Rota	Descrição
GET	/empresa/all?page&size	Lista paginada, cacheada
POST	/empresa	Cria e vincula usuários existentes
GET	/empresa/{id}	Busca por id
GET	/empresa?nome=	Busca por nome
PUT	/empresa	Atualiza
DELETE	/empresa/{id}	Exclui e desvincula usuários

8.5 WebSocket (tempo real)

Canal	Direção	Descrição
/ws (SockJS)	Handshake	Endpoint público de conexão
/topic/pet-location-lote	Servidor → Cliente	Lote de posições a cada 500 ms
/topic/messages	Servidor → Cliente	Canal genérico

9. Front-end Angular

Standalone components (sem NgModule de app): cada tela é um componente autônomo com seus próprios imports.

- Angular Material para UI (cards, tabelas, form fields, selects, ícones, diálogos).
- Leaflet (mapa-pets, pet-mapa-dialog) para exibir o mapa com a localização dos pets.
- @stomp/stompjs + sockjs-client (websocket-service.ts) para consumir os canais em tempo real.

9.1 Rotas (app.routes.ts)

Rota	Componente	Guard
"	redirect → /login	—
/login	Login	loginGuard
/criar-usuario	CriarUsuario	loginGuard
/pets, /pets/novo, /pets/:id	GestaoPets / PetForm	authGuard
/usuarios, /usuarios/:id	GestaoUsuarios / UsuarioForm	authGuard
/empresas, /empresas/novo, /empresas/:id	GestaoEmpresas / EmpresaForm	adminGuard
/mapa	MapaPets	authGuard

Débito resolvido: O adminGuard agora protege de fato as rotas /empresas, /empresas/novo e /empresas/:id.

9.2 Autenticação no front

- auth.interceptor.ts: adiciona withCredentials: true a toda requisição.
- auth.service.ts: login envia e-mail/senha e guarda apenas id/nome/email/role no localStorage; o token fica no cookie httpOnly.
- Guards usam metadados do localStorage para UX; a autorização definitiva é sempre feita pelo back-end.

9.3 Services

- auth.service: login, logout e cadastro.
- usuario.ts: operações de usuários.
- pet.ts: operações de pets e localização.
- empresa.ts: operações de empresas.
- websocket-service.ts: STOMP/SockJS e petLocationBatch\$.
- local-storage-service.ts: wrapper de localStorage.

9.4 Configuração de ambiente

```
// environment.ts
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8080/tela-login-angular'
};

// environment.development.ts
export const environment = {};
```

environment.development.ts continua vazio: O arquivo usado por ng serve isolado está vazio. Como os services dependem de environment.apiUrl, o fluxo funcional continua sendo buildar o Angular e servi-lo pelo Spring Boot. O cookie Secure também favorece testes same-origin/HTTPS.

10. Como Rodar

1. Buildar o Angular: cd angular-frontend && ng build.
2. Copiar os arquivos de angular-frontend/dist para src/main/resources/static.
3. Gerar o WAR: mvn clean package → target/tela-login-angular.war.
4. Copiar o WAR para webapps/ do Tomcat e iniciar o Tomcat (ou usar o script war_para_tomcat.ps1).

Variável de ambiente JWT_SECRET: Defina JWT_SECRET em qualquer ambiente compartilhado. O valor default do application.properties existe apenas para desenvolvimento; em produção, o segredo default permitiria forjar tokens válidos.

11. Banco de Dados

- spring.datasource.url: jdbc:mysql://localhost:3306/tela-login-angular.
- spring.jpa.hibernate.ddl-auto=create: recria o schema a cada start.

- `spring.sql.init.mode=always` + `spring.jpa.defer-datasource-initialization=true`: executa `data.sql` após a criação das tabelas.
- `spring.jpa.show-sql` / `format_sql`: podem ser ativados para depuração.

ddl-auto=create apaga os dados a cada reinício: Com `ddl-auto=create`, os dados são apagados e `data.sql` é reexecutado a cada reinício. Confirme o valor antes de usar em ambiente com dados reais.

O `data.sql` atual popula 1 ADMIN, aproximadamente 23 usuários de teste, 4 pets com localização/peso e 3 empresas.

12. Limitações Conhecidas e Próximos Passos

#	Item	Situação atual
1	Senha em texto puro	Resolvido — BCryptPasswordEncoder
2	Entidade exposta na API	Resolvido — DTOs sem senha
3	Token sem expiração	Parcialmente resolvido — JWT expira em 30 min; blacklist ainda em memória
4	Sem testes automatizados no back-end	Ainda pendente
5	<code>updatePet</code> confiava no dono do payload	Resolvido — troca de dono é rejeitada

12.1 Pontos novos identificados nesta revisão

- Cookie com `.secure(true)` exige HTTPS.
- `environment.development.ts` vazio quebra `ng serve` isolado.
- `JWT_SECRET` possui valor default e deve ser sobrescrito em ambientes reais.
- Caches `SimpleCacheManager` são em memória e por instância.
- `RateLimitingFilter` depende do `context-path` `/tela-login-angular`.
- `UnauthorizedException` usa 401 também em casos que semanticamente seriam 403.